

Aarya Vijay Arban

S11-07

Aim: Write a program for implementation of different operations of Singly Linked List.

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {  
    int data;  
    struct node *next;  
};
```

```
struct node *head = NULL;
```

```
void insert_at_beginning(int data) {  
    struct node *new_node = malloc(sizeof(struct node));  
    new_node->data = data;  
    new_node->next = head;  
    head = new_node;  
}
```

```
void insert_at_end(int data) {  
    struct node *new_node = malloc(sizeof(struct node));  
    new_node->data = data;  
    new_node->next = NULL;
```

```
if (head == NULL) {  
    head = new_node;
```

```
    return;  
}
```

```
struct node *temp = head;  
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

```
temp->next = new_node;  
}
```

```
void insert_at_specified_location(int data, int location) {  
    if (location == 1) {  
        insert_at_beginning(data);  
        return;  
    }
```

```
    struct node *new_node = malloc(sizeof(struct node));  
    new_node->data = data;  
    new_node->next = NULL;
```

```
    struct node *temp = head;  
    int i = 1;  
    while (i < location - 1 && temp != NULL) {  
        temp = temp->next;  
        i++;  
    }
```

```
    if (temp == NULL) {
```

```
    printf("The specified location is out of bounds.\n");  
    return;  
}
```

```
new_node->next = temp->next;  
temp->next = new_node;  
}
```

```
void delete_at_beginning() {  
    if (head == NULL) {  
        printf("The linked list is empty.\n");  
        return;  
    }  
}
```

```
struct node *temp = head;  
head = head->next;  
free(temp);  
}
```

```
void delete_at_end() {  
    if (head == NULL) {  
        printf("The linked list is empty.\n");  
        return;  
    }  
}
```

```
struct node *temp = head;  
while (temp->next->next != NULL) {  
    temp = temp->next;  
}
```

```
    struct node *to_delete = temp->next;
    temp->next = NULL;
    free(to_delete);
}
```

```
void delete_at_specified_location(int location) {
    if (location == 1) {
        delete_at_beginning();
        return;
    }
```

```
    struct node *temp = head;
    int i = 1;
    while (i < location - 1 && temp != NULL) {
        temp = temp->next;
        i++;
    }
```

```
    if (temp == NULL) {
        printf("The specified location is out of bounds.\n");
        return;
    }
```

```
    struct node *to_delete = temp->next;
    temp->next = temp->next->next;
    free(to_delete);
}
```

```
void display() {  
    struct node *temp = head;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
int main() {  
    int choice;  
    int data, location;  
  
    while (1) {  
        printf("\nLinked List Operations\n");  
        printf("1. Insert at beginning\n");  
        printf("2. Insert at end\n");  
        printf("3. Insert at specified location\n");  
        printf("4. Delete at beginning\n");  
        printf("5. Delete at end\n");  
        printf("6. Delete at specified location\n");  
        printf("7. Display\n");  
        printf("8. Exit\n");  
        printf("\nEnter your choice: \n");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                printf("Enter the data to be inserted:\n ");
```

```
scanf("%d", &data);
insert_at_beginning(data);
break;
case 2:
    printf("Enter the data to be inserted:\n ");
    scanf("%d", &data);
    insert_at_end(data);
    break;
case 3:
    printf("Enter the data to be inserted:\n ");
    scanf("%d", &data);

    printf("Enter the location of insertion:\n ");
    scanf("%d", &location);
    insert_at_specified_location(data,location);
break;
case 4:
    delete_at_beginning();
    break;
case 5: delete_at_end();
    break;
case 6:printf("Enter the location of deletion: \n");
    scanf("%d", &location);
    delete_at_specified_location(location);
    break;
case 7:display();
    break;
case 8: printf("Exiting\n");
    return;
```

```
        break;
    default: printf("Invalid choice\n");
}
}
}
```

Output:

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

1

Enter the data to be inserted:

1

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display

8. Exit

Enter your choice:

2

Enter the data to be inserted:

2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

3

Enter the data to be inserted:

3

Enter the location of insertion:

2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning

5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

7

1 3 2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

4

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location

7. Display

8. Exit

Enter your choice:

7

3 2

Linked List Operations

1. Insert at beginning

2. Insert at end

3. Insert at specified location

4. Delete at beginning

5. Delete at end

6. Delete at specified location

7. Display

8. Exit

Enter your choice:

1

Enter the data to be inserted:

1

Linked List Operations

1. Insert at beginning

2. Insert at end

3. Insert at specified location

4. Delete at beginning

5. Delete at end

6. Delete at specified location

7. Display

8. Exit

Enter your choice:

7

1 3 2

Linked List Operations

1. Insert at beginning

2. Insert at end

3. Insert at specified location

4. Delete at beginning

5. Delete at end

6. Delete at specified location

7. Display

8. Exit

Enter your choice:

6

Enter the location of deletion:

2

Linked List Operations

1. Insert at beginning

2. Insert at end

3. Insert at specified location

4. Delete at beginning

5. Delete at end

6. Delete at specified location

7. Display

8. Exit

Enter your choice:

7

1 2

Linked List Operations

1. Insert at beginning

2. Insert at end

3. Insert at specified location

4. Delete at beginning

5. Delete at end

6. Delete at specified location

7. Display

8. Exit

Enter your choice:

5

Linked List Operations

1. Insert at beginning

2. Insert at end

3. Insert at specified location

4. Delete at beginning

5. Delete at end

6. Delete at specified location

7. Display

8. Exit

Enter your choice:

7

1

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

8

Exiting